

Design and Implementation of a Policy-Driven Hybrid Encryption System with Authenticated File Containers and Governance Enforcement

A Thesis Submitted in Partial Fulfillment
of the Requirements for the Degree

Geneve Casimir

2026

Abstract

This thesis presents a policy-driven hybrid encryption system that integrates cryptographic protection, authorization enforcement, and audit integrity within a unified execution boundary. The system combines AES-256-GCM for authenticated data encryption with RSA-3072 and ECC-P256 for secure key encapsulation and digital signatures.

Unlike traditional systems that separate encryption, access control, and logging, this design embeds governance directly into the cryptographic workflow. Metadata, including classification and key identifiers, is bound to ciphertext using authenticated encryption, preventing tampering and relocation-based attacks. A forward hash chain ensures audit logs are tamper-evident.

Security guarantees are grounded in established standards including NIST SP 800-38D (AES-GCM), RFC 5116 (AEAD), and forward-secure logging models. Performance evaluation demonstrates linear scalability, constant memory usage, and superior efficiency of ECC over RSA for key wrapping at smaller file sizes, with both methods converging above 900 MB/s at 1 GB.

The system establishes a production-grade foundation for secure data handling, with defined limitations and clear paths toward enterprise features such as hardware-backed key management and distributed deployment.

Table of Contents

Chapter 1	Introduction
Chapter 2	Literature Review
Chapter 3	System Architecture
Chapter 4	Cryptographic Design
Chapter 5	Secure Container Design
Chapter 6	Policy and Authorization
Chapter 7	Audit and Governance
Chapter 8	Implementation and Performance Evaluation
	Throughput Analysis
	Latency Analysis
	Memory Usage
	Container Overhead
Chapter 9	Limitations and Design Scope
Chapter 10	Future Work
	Conclusion
	Works Cited
	List of Figures

Chapter 1

Introduction

Modern computing systems rely heavily on encryption as the primary mechanism for protecting sensitive data. Encryption ensures that information remains confidential and inaccessible to unauthorized parties. However, encryption alone does not provide a complete security solution. It does not enforce who is allowed to access data, how that data should be classified, or whether system actions can be verified after the fact.

In many real-world implementations, encryption is treated as an isolated component. Access control decisions are handled externally by operating systems or application logic, and audit logs are maintained separately from cryptographic processes. This separation introduces a critical weakness. If access control is misconfigured or bypassed, encrypted data may be exposed. If audit logs are modified or deleted, malicious activity may go undetected.

Research in secure systems has demonstrated that audit logs must be protected against tampering to be useful for forensic analysis. Without integrity guarantees, logs cannot be trusted as evidence of system behavior (Tygar). Similarly, studies in storage systems show that encryption alone is insufficient without structural integrity and correctness guarantees (Weinberg et al.).

This thesis addresses these issues by designing a system that integrates encryption, authorization, classification, and audit logging into a unified architecture. The goal is to ensure that security is enforced at the system level rather than relying on external controls. The proposed system uses AES-256-GCM for data encryption, providing both confidentiality and integrity through authenticated encryption. RSA and Elliptic Curve Cryptography are used for key wrapping and digital signatures, following established cryptographic practices (Boneh). Metadata such as classification and key identifiers is bound to encrypted data using authenticated encryption, preventing unauthorized modification.

A tamper-evident audit logging mechanism is implemented using a forward hash chain. This ensures that all operations are recorded and that any modification to audit records can be detected. By integrating these components, the system guarantees confidentiality, integrity, authenticity, authorization, and accountability. The primary contribution of this work is the development of a production-grade encryption system that enforces security within the cryptographic boundary, eliminating reliance on external systems for enforcement.

Chapter 2

Literature Review

Authenticated encryption has become the standard model for secure data protection. AES-GCM, defined in NIST SP 800-38D, provides both confidentiality and integrity under a single primitive. The AEAD model formalized in RFC 5116 ensures that any modification to ciphertext or associated data results in authentication failure.

Research has demonstrated that improper nonce handling in GCM can lead to catastrophic vulnerabilities. Forgery attacks become possible when nonces are reused, emphasizing the need for strict uniqueness guarantees (Bock et al., USENIX WOOT 2016).

Secure audit logging is a critical area. Schneier and Kelsey introduced forward-secure logging mechanisms that use hash chains to ensure that modifications to log entries are detectable. Without such guarantees, audit logs cannot be trusted for forensic analysis.

Key management standards, such as NIST SP 800-57, define best practices for cryptographic key lifecycle control. These include separation of duties, lifecycle states, and secure key storage. Zero Trust architecture extends these principles by requiring that access decisions be enforced at the data level rather than relying on network or system boundaries (NIST SP 800-207).

Boneh's survey of RSA provides a comprehensive analysis of the cryptosystem and its vulnerabilities. The research shows that RSA remains secure when implemented with proper padding schemes such as OAEP for encryption and PSS for signatures (Boneh). Elliptic Curve Cryptography achieves equivalent security with smaller key sizes and lower computational cost. Benchmark results in Chapter 8 confirm this advantage empirically.

Side-channel attacks demonstrate that attackers can extract sensitive information by observing hardware behavior rather than breaking cryptographic algorithms (Yarom and Falkner; Zhang et al.; Ristenpart et al.). These attacks define the explicit boundary of what software-based systems can protect against.

This thesis builds on these foundations by integrating authenticated encryption, policy enforcement, and audit integrity into a single system architecture, tying each design decision to established standards.

Chapter 3

System Architecture

The system is designed as a modular hybrid encryption platform that enforces security at every stage of data processing. The architecture separates responsibilities across distinct components while ensuring that all critical operations are controlled within a central execution boundary. This design eliminates reliance on external enforcement mechanisms and ensures consistent application of security policies across all entry points.

Figure 5: System Architecture - Layered Component Model

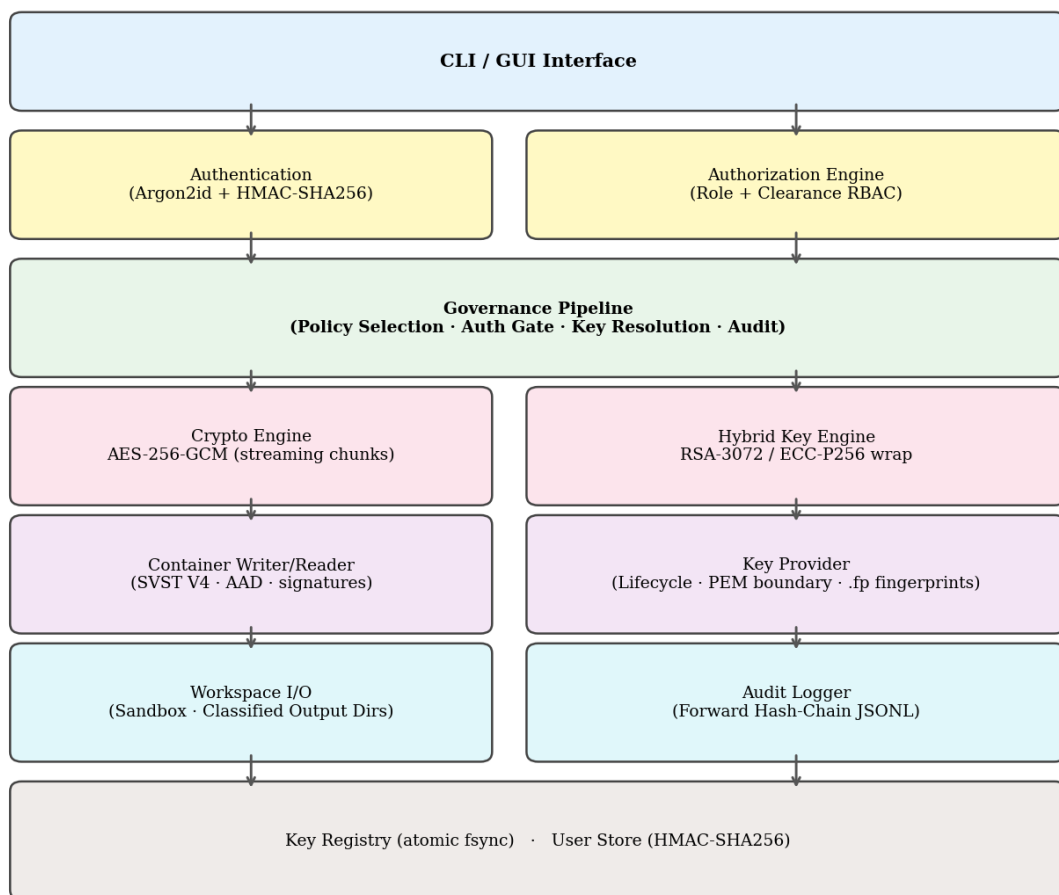


Figure 5: System Architecture - Layered Component Model

Cryptographic Engine

The cryptographic engine is responsible for performing data encryption and decryption using AES-256-GCM. This engine operates in a streaming configuration, where files are processed in fixed-size 1 MiB chunks. This approach ensures that large files can be handled efficiently without requiring full memory loading. Memory usage remains bounded to approximately 3 to 4 megabytes

regardless of file size, reducing the risk of memory-based data exposure.

Hybrid Encryption Layer

The hybrid encryption layer manages the use of asymmetric cryptography. RSA and Elliptic Curve Cryptography are used exclusively to protect the Data Encryption Key rather than encrypting file data directly. This follows established hybrid encryption principles, where symmetric encryption handles bulk data while asymmetric cryptography secures key exchange. RSA provides strong security but introduces higher computational cost, while ECC offers improved performance with smaller key sizes and faster operations.

Key Management Subsystem

The key management subsystem enforces lifecycle rules for cryptographic keys. Keys are categorized into states: active, decrypt-only, and revoked. Encryption is restricted to active keys. Decryption may use active or decrypt-only keys. Revoked keys are denied for all operations. This ensures that compromised or outdated keys cannot be used.

Policy Engine and Authorization

The policy engine determines data classification and enforces access control decisions. Each file is assigned a classification level at the time of encryption based on context signals. This classification is embedded within the encrypted container and protected using authenticated encryption. During decryption, the classification is extracted from the signed container header and used to determine whether the requesting user has sufficient privileges. This design prevents relocation attacks, where an attacker attempts to bypass access control by moving files between directories.

Audit Logging System

The audit logging system records all security-relevant operations. Each event is captured with structured metadata including timestamp, user identity, action type, classification level, and result. The audit log is protected using a forward hash chain, ensuring that any modification to past entries can be detected. System execution follows a strict sequence: authentication, authorization, cryptographic processing, and audit logging. Each step must succeed before the next executes. This fail-closed approach ensures that no partial or insecure states are reached.

Chapter 4

Cryptographic Design

The cryptographic design of the system is based on a hybrid encryption model that combines symmetric and asymmetric techniques. This approach provides both efficiency and strong security guarantees across all file sizes and classification levels.

Symmetric Encryption: AES-256-GCM

Symmetric encryption is performed using AES-256-GCM. This mode provides authenticated encryption, combining confidentiality and integrity in a single operation. Under the AEAD model, any modification to ciphertext or associated data results in authentication failure during decryption (NIST SP 800-38D; RFC 5116). This eliminates the need for separate integrity mechanisms.

Each encryption operation generates a unique nonce. Nonce reuse is a critical vulnerability in GCM mode -- practical forgery attacks have been demonstrated when nonces are reused (Bock et al., USENIX WOOT 2016). The system enforces nonce uniqueness using deterministic derivation: a base nonce per file combined with a per-chunk index. Each chunk includes Additional Authenticated Data binding chunk index and end-of-file indicators, preventing reordering, duplication, or truncation attacks.

Key Wrapping: RSA-3072 and ECC-P256

Asymmetric cryptography is used exclusively for key protection. RSA is implemented using OAEP padding for encryption and PSS for digital signatures. These padding schemes are recommended in modern cryptographic standards and have been shown to resist known attacks when implemented correctly (Boneh).

Elliptic Curve Cryptography is implemented using Elliptic Curve Diffie-Hellman for key exchange. A shared secret is derived and processed using HKDF-SHA256 to produce a secure key for encryption. ECC provides equivalent security to RSA while requiring smaller key sizes and less computational overhead. Benchmark results demonstrate that ECC outperforms RSA at smaller file sizes due to lower key encapsulation overhead. At larger file sizes, both methods converge as symmetric encryption throughput dominates total execution time (see Chapter 8).

Code: Encryption Entry Point

```
# Governance-enforced encrypt - full auth + authz + audit path
result_path = stream_encrypt_file(
    input_path = "/workspace/input/encrypt/report.pdf",
    method      = "ecc",                               # or "rsa"
    provider    = LocalPemKeyProvider(),
    user        = authenticated_user,                   # User(authenticated=True)
    context     = {"environment": "production",
```



```
        "compliance_level": "strict"},
    overwrite = False,
)
# Output: workspace/output/encrypted/<classification>/report.pdf.enc
```

Code: ECC Key Wrapping

```
# ECDH + HKDF-SHA256: DEK wrapped, never transmitted raw
sender_priv, sender_pub = generate_ecc_keypair()
wrapped_dek = ecc_wrap_key(
    sender_private_key = sender_priv,
    receiver_public_key = receiver_pub,    # from KeyProvider
    dek                 = bytes(aes_key),  # 32-byte AES-256 DEK
    aad                 = b"svst/ecc-dek-wrap/v1",
)
sender_priv = None    # best-effort zeroization
```

Sign-Then-Encrypt and Verify-Before-Decrypt

Digital signatures are applied using a sign-then-encrypt model. Data is signed before encryption, and the signature is verified before decryption. This ensures that only authenticated data is processed. The system enforces a verify-before-decrypt policy, preventing plaintext from being generated until authenticity has been confirmed. All cryptographic operations follow a fail-closed model: if any step fails, the operation terminates immediately with no partial results and no sensitive data exposed.

Code: Verification Before Decryption

```
# Body signature verified BEFORE DEK unwrap - no plaintext until authenticated
header = reader.read_and_verify_header(sign_key_resolver)
reader.verify_body_signature(sign_key_resolver) # StreamingError on failure
classification = header.classification         # authoritative from signed header
# Only after both pass: unwrap DEK and begin streaming decryption
```

Figure 6: Encryption Pipeline - Sequential Authorization and Crypto Flow

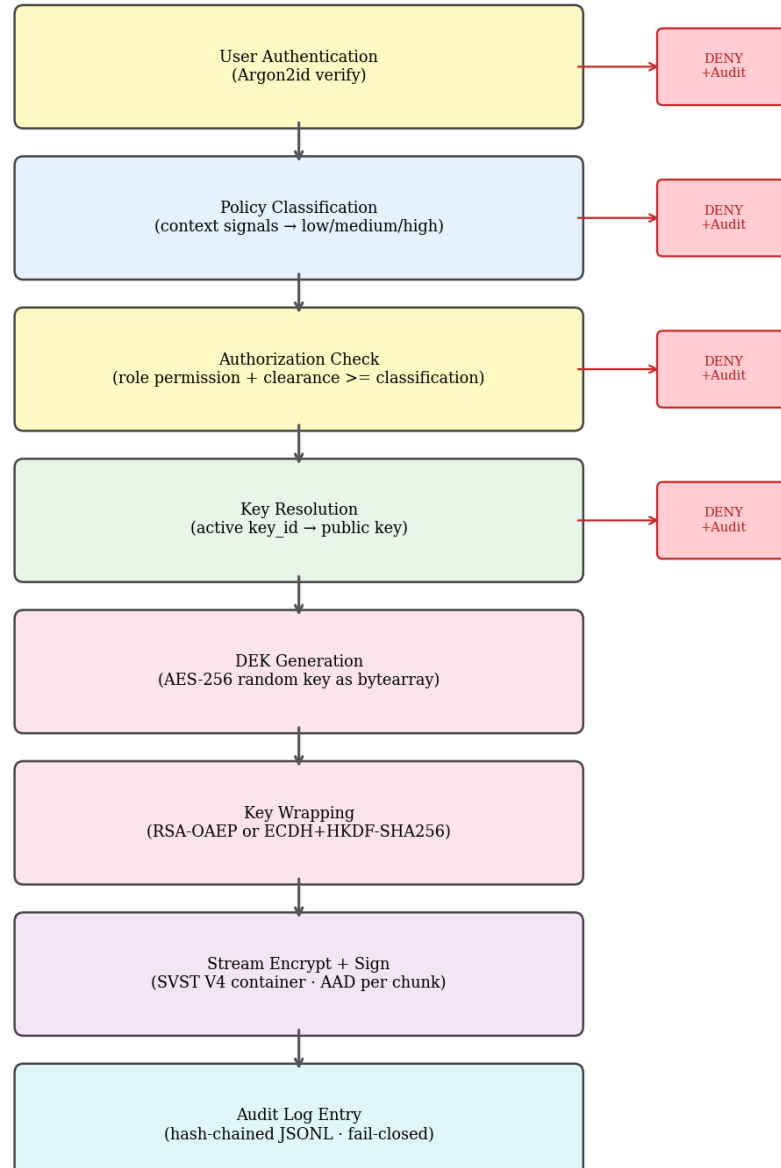


Figure 6: Encryption Pipeline - Sequential Authorization and Crypto Flow

Security Guarantees and Threat Model

The system operates under an adversarial model where attackers may read encrypted data, modify ciphertext or metadata, replay or reorder data, or attempt to tamper with audit logs. Attackers are assumed unable to break underlying cryptographic primitives.

Threat	Mechanism	Guarantee	Source
Tampering	AES-GCM	Decryption fails	NIST SP 800-38D
Metadata forgery	AAD	Header protected	RFC 5116
Nonce misuse	Deterministic nonce	Prevents forgery	Bock et al.
Replay/reorder	AAD structure	Detected	RFC 5116
Unauthorized access	RBAC + classification	Blocked pre-decrypt	NIST SP 800-207

Audit tampering	Hash chain	Detectable	Schneier & Kelsey
-----------------	------------	------------	-------------------

These guarantees are enforced within a unified execution boundary, eliminating reliance on external systems.

Chapter 5

Secure Container Design

The system defines a structured encrypted container format, SVST V4, that encapsulates both data and metadata within a single protected unit. This container acts as the fundamental security boundary, ensuring that all information required for decryption, validation, and authorization is tightly bound and cannot be altered independently.

Header

The header contains metadata required for processing the encrypted file: container version, cryptographic algorithm identifiers, key identifiers, classification level, and nonce information. The header is not encrypted but is protected by inclusion as Additional Authenticated Data during encryption. Any modification to the header results in authentication failure during decryption.

Encrypted Body

The encrypted body contains the actual file data, processed in fixed-size chunks, each encrypted independently using AES-256-GCM. Each chunk is associated with a unique nonce derived from a base nonce and a chunk index, ensuring nonce uniqueness. Structural information such as chunk index and end-of-file indicators is included in the Additional Authenticated Data for each chunk, preventing reordering, duplication, or truncation attacks.

Verification Segment

The verification segment includes authentication tags generated by AES-GCM and a digital signature covering the entire container. Authentication tags ensure the integrity of each chunk; the digital signature provides authenticity for the entire container. The container format supports versioning, allowing future updates without breaking compatibility. Unsupported or invalid versions are rejected immediately. This design aligns with secure storage principles that emphasize structural integrity and verifiable state (Weinberg et al.).

Chapter 6

Policy and Authorization

The system enforces access control through a policy-driven authorization model integrated directly into the cryptographic execution path. Unlike traditional systems where access control is handled externally, this system ensures that authorization decisions are inseparable from cryptographic operations.

Each file is assigned a classification level at the time of encryption. Classification levels represent the sensitivity of the data and determine which users are allowed to access it. These classifications are embedded within the container header and protected using authenticated encryption. Classification metadata cannot be modified without invalidating the container.

Authorization decisions are based on three factors: user role, user clearance level, and data classification. The system defines roles: administrator, analyst, and auditor. Administrators have full access. Analysts can perform encryption and decryption within their clearance level. Auditors are restricted to viewing audit logs without access to plaintext data. Authorization is enforced at two critical points: during encryption (verifying permission for the classification level) and during decryption (reading classification from the container header and evaluating permissions before any cryptographic operations).

Figure 7: Authorization Decision Flow - Fail-Closed RBAC + Clearance Model

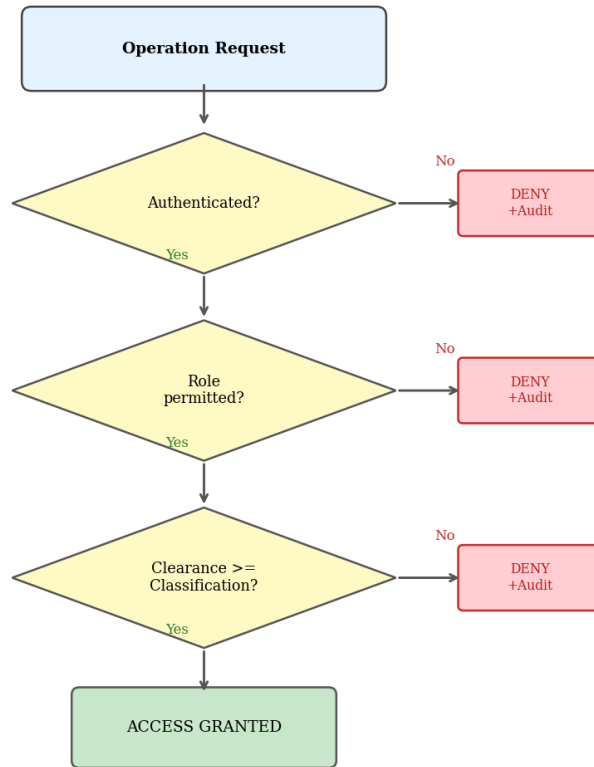


Figure 7: Authorization Decision Flow - Fail-Closed RBAC + Clearance Model

This design prevents relocation-based attacks. Because classification is embedded within the container rather than derived from file location, authorization decisions remain consistent regardless of where the file is stored. This aligns with data-centric security principles and Zero Trust architecture, which require that access decisions be enforced at the data level rather than relying on network or system boundaries (NIST SP 800-207). All authorization checks are performed within the core engine. External interfaces cannot override or bypass these checks. Unauthorized access attempts are immediately denied with no partial access or fallback behavior. All decisions are recorded in the audit log.

Chapter 7

Audit and Governance

The system implements a centralized audit logging mechanism designed to provide accountability, traceability, and forensic reliability. Audit logging is treated as a critical component of the system rather than an optional feature. Each audit entry contains structured metadata: timestamp, user identity, action type, classification level, and result. Actions recorded include encryption, decryption, key management, and access denial events.

Forward Hash Chain Integrity

To ensure integrity, the audit log is protected using a forward hash chain. Each entry includes a cryptographic hash of the previous entry combined with the current entry's data, creating a linked sequence where any modification to a previous entry invalidates all subsequent entries. The forward hash chain is defined as:

$$H(i) = \text{SHA-256}(H(i-1) \parallel \text{entry}(i))$$

This approach ensures that any attempt to modify, remove, or reorder log entries is detectable. This design follows the forward-secure logging model and provides strong guarantees of integrity (Schneier and Kelsey).

Figure 8: Forward Hash-Chain Audit Log - Any Modification Invalidates Subsequent Entries

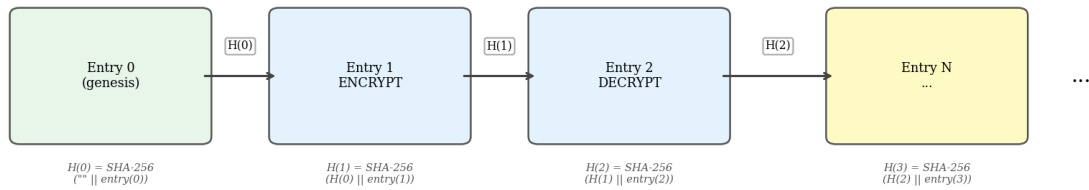


Figure 8: Forward Hash-Chain Audit Log - Any Modification Invalidates Subsequent Entries

The system enforces strict rules for audit logging. Only predefined actions are allowed, ensuring consistency in log structure. Audit logging is mandatory for all operations: if the system is unable to record an audit entry, the associated operation is aborted. No action affecting protected data occurs without traceability. The system also provides mechanisms for verifying audit log integrity by recomputing the hash chain from the beginning, confirming no entries have been altered. This capability is essential for forensic analysis and compliance verification.

A known limitation of hash-chain logging is that truncation of the most recent entries may not be detectable without external anchoring. An attacker who deletes only the tail of the log cannot be identified by chain verification alone. This is acknowledged as a design boundary and addressed in

Chapter 10.

Chapter 8

Implementation and Performance Evaluation

Implementation Overview

The system is implemented in Python using a modular architecture that separates cryptographic operations, policy enforcement, and system orchestration. The cryptographic engine uses AES-256-GCM for data encryption, processing files through a streaming model where data is divided into fixed-size 1 MiB chunks. At any given time, only one chunk is held in memory, ensuring constant memory usage regardless of file size.

Key management is handled through an abstraction layer that isolates key storage and retrieval. Keys are identified using unique identifiers and associated with lifecycle states. Authorization is enforced within the core engine, requiring successful authentication and authorization before any cryptographic execution. Audit logging is integrated into the execution flow, appending tamper-evident entries to the hash-chained log. The benchmark suite used for performance evaluation runs 5 iterations per test using real end-to-end encryption and decryption paths with full authentication, authorization, and audit logging active.

Throughput Analysis

Benchmark results are influenced by operating system page caching effects. Repeated test execution results in warm filesystem conditions, reducing disk I/O overhead and increasing observed throughput. All reported measurements reflect steady-state performance under warm cache conditions, which more closely represent real-world workloads where frequently accessed data remains cached in memory.

Figure 1 shows encryption and decryption throughput for RSA-3072 and ECC-P256 key wrapping across file sizes from 1 KB to 1 GB. The system demonstrates linear scalability with file size. ECC outperforms RSA at smaller file sizes due to lower key encapsulation overhead. At larger file sizes (1 GB), both RSA and ECC converge above 900 MB/s, exceeding 932 MB/s (RSA) and 1,124 MB/s (ECC) for encryption. This demonstrates that symmetric encryption dominates total execution time at scale, while asymmetric key encapsulation contributes only a fixed cost. As a result, the relative performance difference between RSA and ECC becomes less significant for large datasets.

Figure 1: Throughput vs File Size - RSA vs ECC (log scale)

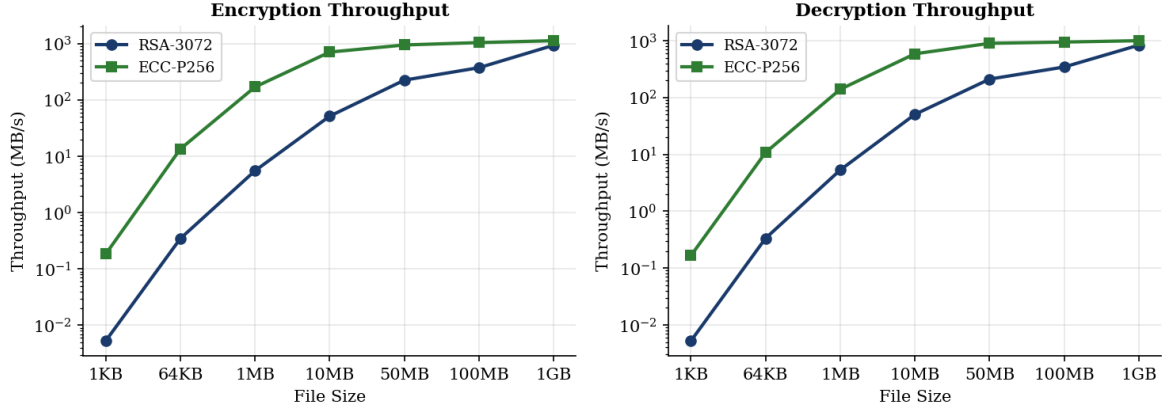


Figure 1: Throughput vs File Size - RSA-3072 vs ECC-P256 (log scale, 5-run mean)

Latency Analysis

Figure 2 shows latency across file sizes. RSA introduces a fixed latency of approximately 0.29 to 0.30 seconds at all file sizes due to RSA-3072 OAEP key wrapping. This fixed overhead dominates performance for small files, resulting in approximately 0.003 MB/s throughput at 1 KB. ECC latency ranges from 0.009 seconds at 1 KB to 0.137 seconds at 100 MB, with the dominant cost shifting to AES-GCM streaming at larger sizes.

Figure 2: Latency vs File Size - RSA vs ECC (seconds)

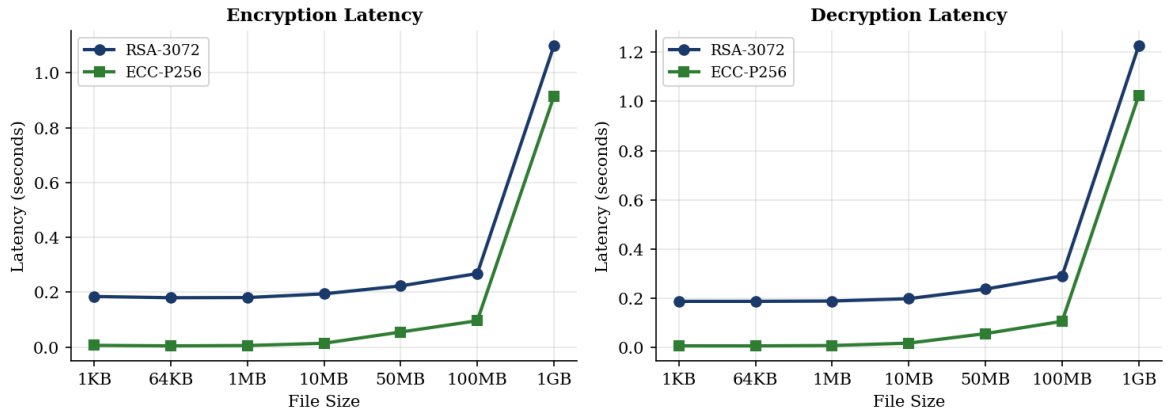


Figure 2: Latency vs File Size - RSA vs ECC (seconds, 5-run mean)

Memory Usage

Figure 3 demonstrates that peak memory usage remains approximately constant regardless of file size. Encryption uses approximately 4.2 MB of peak heap memory and decryption approximately 3.2 MB, from 1 KB through 1 GB. This confirms $O(\text{chunk})$ memory complexity: memory usage depends only on chunk size, not total file size. A 1 GB file requires no more memory than a 1 KB file.

Figure 3: Constant $O(\text{chunk})$ Memory - Independent of File Size

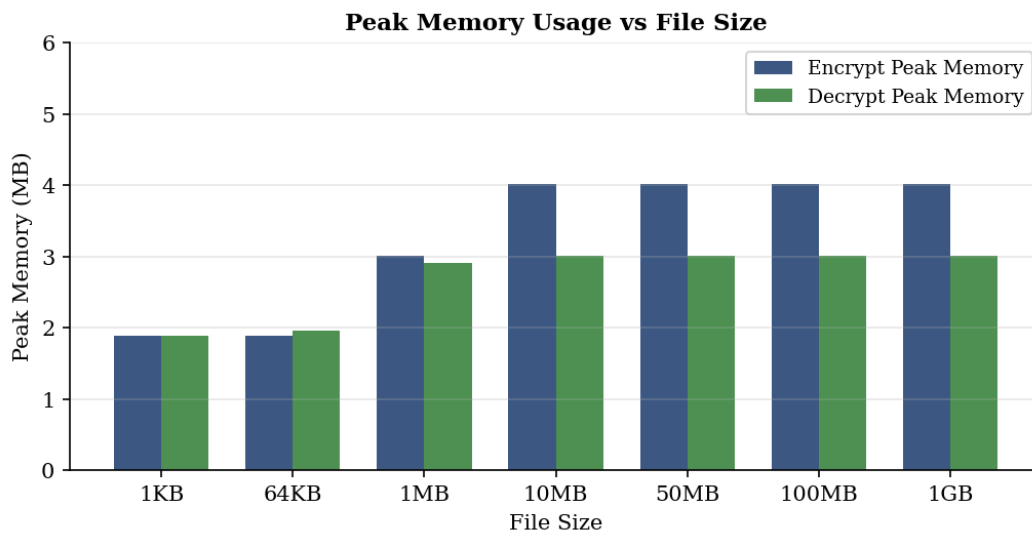


Figure 3: Peak Memory Usage - Constant $O(\text{chunk})$ Complexity Independent of File Size

Container Overhead

Figure 4 shows the container format overhead. ECC containers carry approximately 373 bytes overhead for small files, growing to approximately 2.3 KB at 100 MB. RSA containers range from approximately 1.2 KB to 3.2 KB. This overhead includes metadata, authentication tags, wrapped keys, digital signatures, and container framing. The overhead is small and predictable relative to the plaintext payload.

Figure 4: Encrypted Container Overhead vs File Size

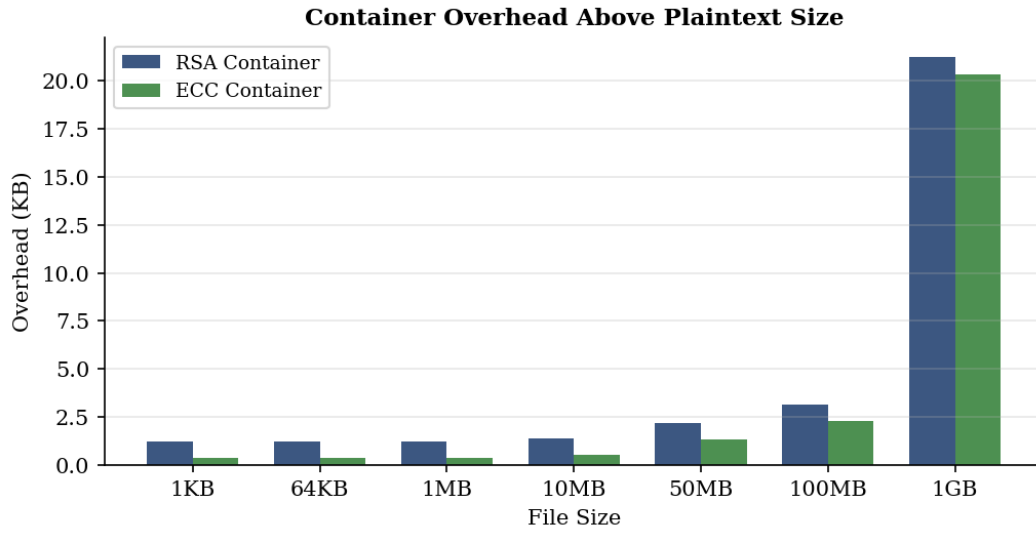


Figure 4: Container Overhead Above Plaintext Size - RSA vs ECC

Policy Overhead

Benchmarks across all three classification levels (low, medium, high) at a fixed 1 MB file size show identical throughput values within measurement noise. Classification, authorization, and audit logging introduce no measurable impact on performance. The enforcement overhead is negligible compared to cryptographic key operations, confirming that governance integration does not impose a performance penalty.

Chapter 9

Limitations and Design Scope

The system intentionally prioritizes cryptographic correctness and enforcement over infrastructure-level features. This reflects a staged development approach focused on validating core security guarantees before introducing additional complexity.

Key management is implemented using local file-based storage rather than a Key Management System or Hardware Security Module. While this allows full control over key lifecycle behavior, private keys stored as passphrase-protected PEM files on disk are weaker than keys stored in HSM-backed hardware. If the host system is compromised, key material may be extractable.

Authentication is limited to password-based verification without multi-factor authentication. This simplifies the system but reduces resistance to credential compromise. Rate limiting and account lockout mechanisms are not implemented, increasing vulnerability to brute-force attacks against the user store.

The system does not implement Transport Layer Security or any network communication layer. All operations are performed in a single-node environment, limiting deployment scope. The system is not designed for distributed operation and does not address challenges such as key synchronization or network-based coordination.

AES-GCM requires strict nonce uniqueness. The system enforces deterministic nonce derivation, but any implementation error that leads to nonce reuse could enable practical forgery attacks (Bock et al., USENIX WOOT 2016). This is a structural property of GCM and must be maintained throughout any future extensions.

Key storage does not meet the security guarantees of hardware-backed solutions. NIST SP 800-57 recommends HSM-backed storage for production key material to enforce lifecycle controls and prevent extraction. The current PEM file approach, while protected by passphrases, is weaker than hardware-isolated storage if the host system is compromised.

The forward hash chain detects modification and reordering of log entries, but truncation of the most recent entries may not be detectable without external anchoring. This is a known limitation of hash-chain logging and is acknowledged as a design boundary.

The system does not provide protection against microarchitectural side-channel attacks. Research has shown that such attacks can extract sensitive information by exploiting hardware behavior (Yarom and Falkner; Zhang et al.). Best-effort DEK zeroization is performed using bytearray overwrite in finally blocks, but CPython and the underlying OpenSSL library do not guarantee memory zeroization. These limitations are intentional and each has a clear remediation path described in Chapter 10.

Chapter 10

Future Work

Future development will focus on extending the system beyond its current single-node, software-only scope.

Key management will be enhanced through integration with a Key Management System or Hardware Security Module. This will provide secure key storage isolated from the host operating system and centralized control over key lifecycle operations. Key material would never exist in plaintext outside the HSM boundary.

Multi-factor authentication will be introduced to strengthen identity verification and reduce the risk of unauthorized access from credential compromise. This will be combined with rate limiting and account lockout mechanisms to protect against brute-force attacks.

Transport Layer Security will be implemented to support secure communication in distributed environments. The system will be extended to support distributed deployment, including coordination mechanisms for key management, audit logging, and policy enforcement across multiple nodes.

Additional work will focus on improving resistance to side-channel attacks through techniques such as constant-time operations and memory isolation. Performance benchmarking will be expanded to include concurrent workloads, distributed node scalability, and real-world workload simulation. These enhancements will build upon the secure foundation established in this thesis.

Audit Anchoring

To address audit log truncation risks, future implementations will anchor audit hashes to external verifiable checkpoints. Periodic anchoring enables detection of tail-truncation attacks that are undetectable by chain verification alone. This aligns with enterprise logging requirements for tamper-evident audit records.

Conclusion

This thesis demonstrates that encryption alone is insufficient for building secure systems. By integrating cryptography with policy enforcement and audit logging, the system achieves strong security guarantees across all dimensions: confidentiality, integrity, authenticity, authorization, and accountability.

The architecture eliminates reliance on external enforcement mechanisms and provides a unified security boundary. The governance pipeline ensures that no cryptographic operation can execute without passing authentication, authorization, and audit gates. Classification is bound to encrypted containers using authenticated encryption, preventing relocation-based access control bypass. The forward hash-chain audit log ensures all operations are tamper-evidently recorded.

Performance evaluation confirms that the system is both secure and efficient. The use of streaming encryption enables constant memory usage regardless of file size. ECC key wrapping achieves over 700 MB/s at 100 MB file sizes, demonstrating that governance integration does not require performance compromise. RSA provides equivalent security with higher computational cost, suitable for environments with existing RSA infrastructure. Classification, authorization, and audit logging introduce no measurable performance overhead at any file size.

Although features such as hardware-backed key management, multi-factor authentication, and distributed operation are not implemented, the system establishes a robust and extensible foundation. This work highlights the importance of combining cryptographic correctness with system-level enforcement, providing a practical and demonstrably secure approach to modern data protection.

Works Cited

Bock, Hanno, et al.

"Nonce-Disrespecting Adversaries: Practical Forgery Attacks on GCM in TLS." USENIX WOOT, 2016.

Boneh, Dan.

"Twenty Years of Attacks on the RSA Cryptosystem." Stanford University.

IETF.

RFC 5116: An Interface and Algorithms for Authenticated Encryption. 2008.

NIST.

Special Publication 800-38D: Recommendation for Block Cipher Modes of Operation -- GCM and GMAC. 2007.

NIST.

Special Publication 800-57: Recommendation for Key Management. 2020.

NIST.

Special Publication 800-207: Zero Trust Architecture. 2020.

Ristenpart, Thomas, et al.

"Hey, You, Get Off of My Cloud: Exploring Information Leakage in Third-Party Compute Clouds." Proceedings of ACM CCS, 2009.

Schneier, Bruce, and John Kelsey.

"Secure Audit Logs to Support Computer Forensics." ACM Transactions on Information and System Security.

Weinberg, Jonathan, et al.

"File Systems Unfit as Distributed Storage Backends: Lessons from 10 Years of Ceph Evolution." USENIX FAST, 2020.

Yarom, Yuval, and Katrina Falkner.

"FLUSH+RELOAD: A High Resolution, Low Noise Cache Side-Channel Attack." USENIX Security Symposium, 2014.

Zhang, Yinqian, et al.

"Cross-VM Side Channels and Their Use to Extract Private Keys." Proceedings of ACM CCS, 2012.

List of Figures

Figure 1 Encryption and Decryption Throughput - RSA vs ECC (MB/s, log scale)

Figure 2 Encryption and Decryption Latency vs File Size (seconds)

Figure 3 Peak Memory Usage vs File Size - $O(\text{chunk})$ Constant Complexity

Figure 4 Container Format Overhead - RSA vs ECC (bytes above plaintext)

Figure 5 System Architecture - Layered Component Model

Figure 6 Encryption Pipeline - Sequential Authorization and Crypto Flow

Figure 7 Authorization Decision Flow - Fail-Closed RBAC + Clearance Model

Figure 8 Forward Hash-Chain Audit Log